

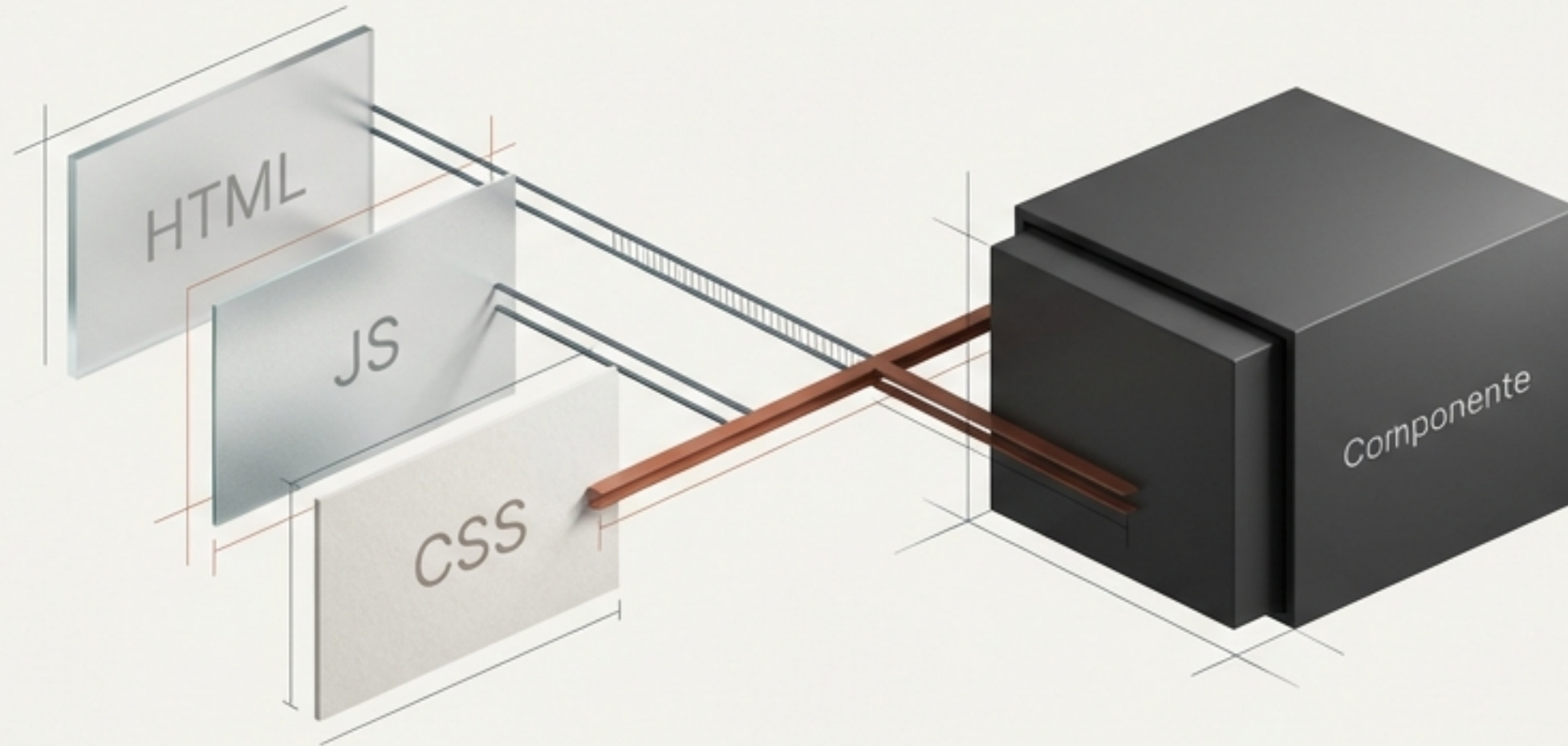
A Arte da Arquitetura de UI

Dominando a fundação
dos componentes React.

O Bloco de Construção Fundamental

No passado, a web separava conteúdo, design e lógica em arquivos diferentes. O React unifica tudo isso em um único bloco encapsulado e reutilizável: o Componente.

Um componente React é, em sua essência, uma função JavaScript perfeitamente lapidada que retorna uma interface de usuário.



```
export default function Profile() { ... }
```


JSX Como Planta Baixa

Para desenhar nossos blocos, utilizamos o JSX: uma extensão de sintaxe que permite escrever marcação semelhante ao HTML dentro do JavaScript.

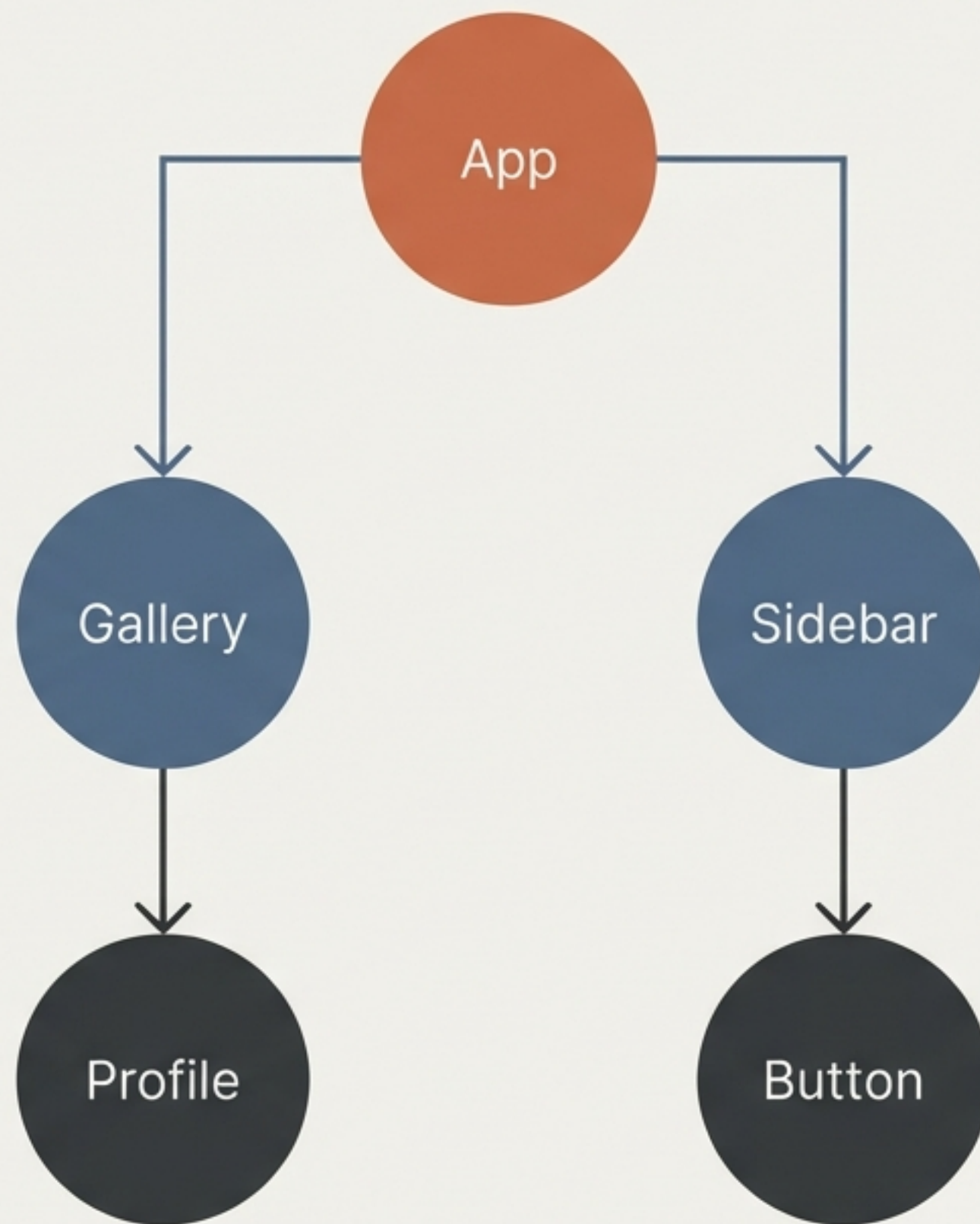
1. **Uma única raiz:** Múltiplos elementos devem ser agrupados por uma tag pai.
2. **Fechamento estrito:** Todas as tags devem ser fechadas. Uma imagem não é apenas `<img>`, é `<img />`.
3. **Padrão camelCase:** Atributos adotam o camelCase. Em vez de **class**, usamos **className**.

```
<>  
  <h1>Título</h1>  
  <img src='foto.jpg'  
  className='perfil' />  
</>
```


A Árvore Renderizada

A arquitetura de uma aplicação React não é uma lista plana; é uma estrutura hierárquica. O React modela as relações entre os componentes como uma Árvore de Renderização.

- **Componentes de Topo (Raiz):** Ditam a estrutura principal e propagam dados para baixo.
- **Componentes Folha:** Elementos isolados na base da árvore que retornam apenas marcação visual.



Props Como Interfaces

Componentes são independentes, mas precisam se comunicar. Eles fazem isso através de Props (propriedades).

Pense nas props como os botões de ajuste de uma máquina. Um componente pai passa informações para um componente filho, configurando como ele deve ser exibido.

A Regra de Ouro das Props:

Elas são instantâneos de dados no tempo. As props são 100% imutáveis. Um componente nunca deve tentar modificar as props que recebe.

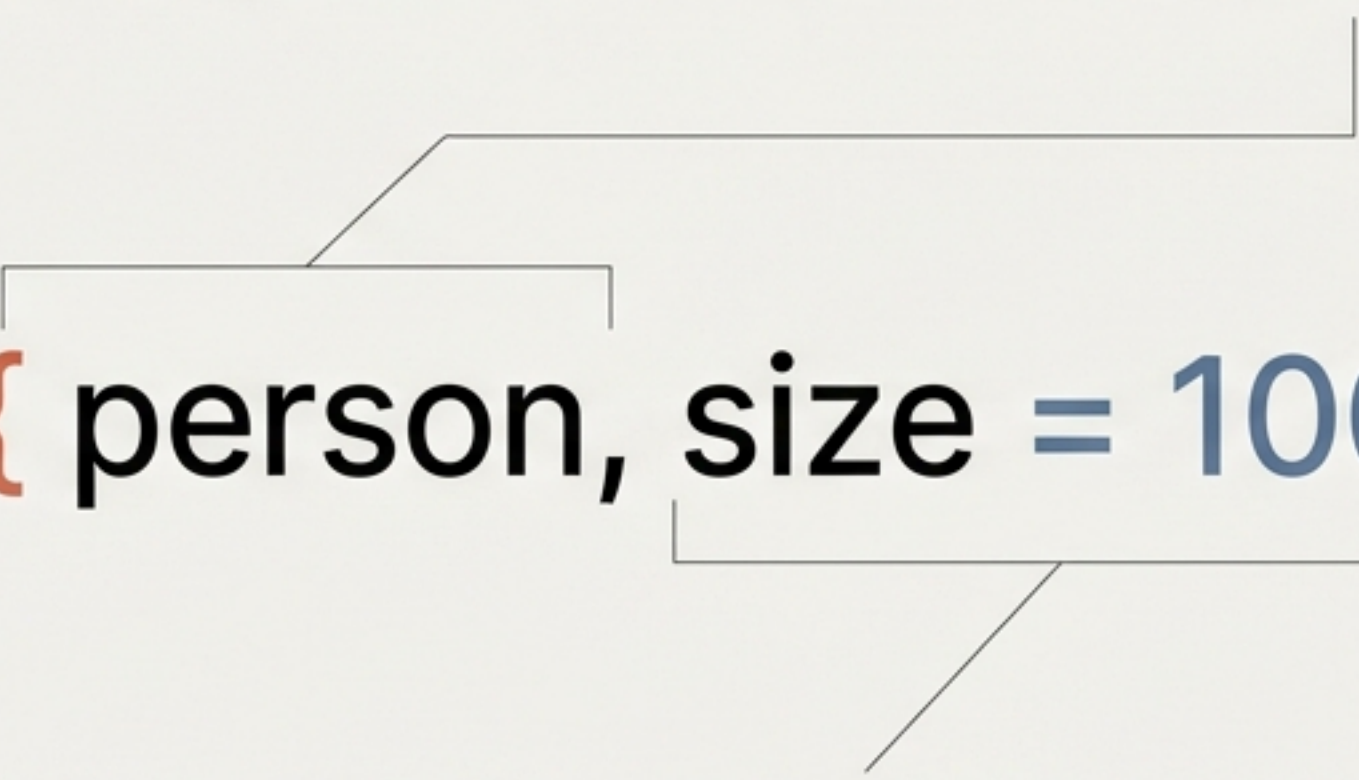


```
<Avatar person={userData}  
size={100} />
```


Extração e Valores Seguros

Interfaces limpas são fáceis de ler e difíceis de quebrar. Em vez de acessar `props.person` repetidamente, extraímos os valores diretamente na assinatura da função usando Destructuring.

Também podemos definir valores padrão para garantir que a arquitetura não quebre se um dado estiver ausente.



```
function Avatar( { person, size = 100 } ) { ... }
```

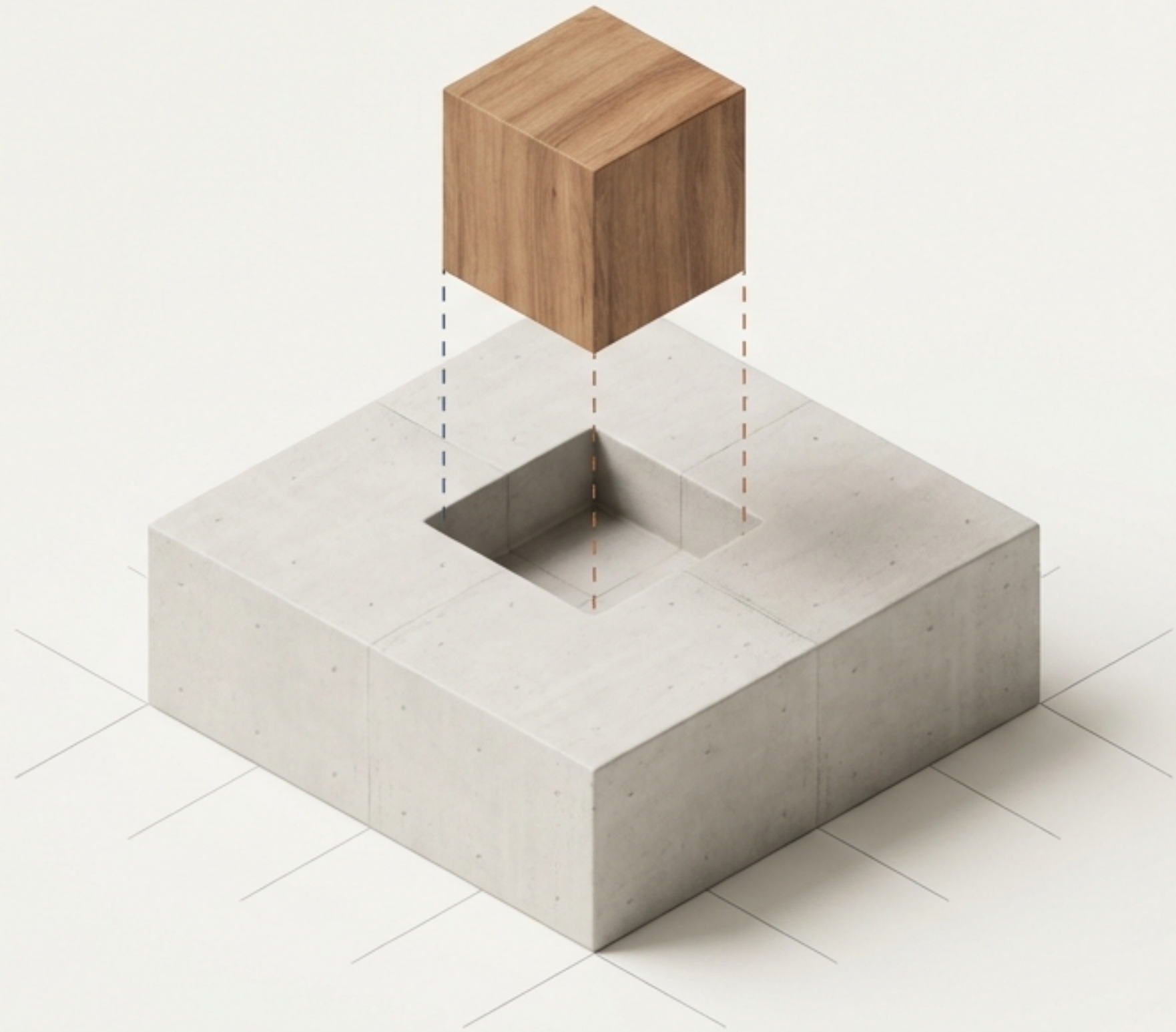
Se o pai renderizar `<Avatar person={userData} />` sem definir o tamanho, a arquitetura se protege assumindo o valor seguro de 100.

O Encaixe Flexível

Alguns componentes não precisam saber o que vão renderizar em seu interior. Eles funcionam como invólucros visuais (como Painéis, Cards ou Layouts).

O React permite passar componentes inteiros para dentro de outros através de uma prop especial chamada **children**. É como projetar um bloco com um espaço vazio no meio, pronto para ser preenchido pelo pai.

```
<Card>  
  <Avatar />  
</Card>
```

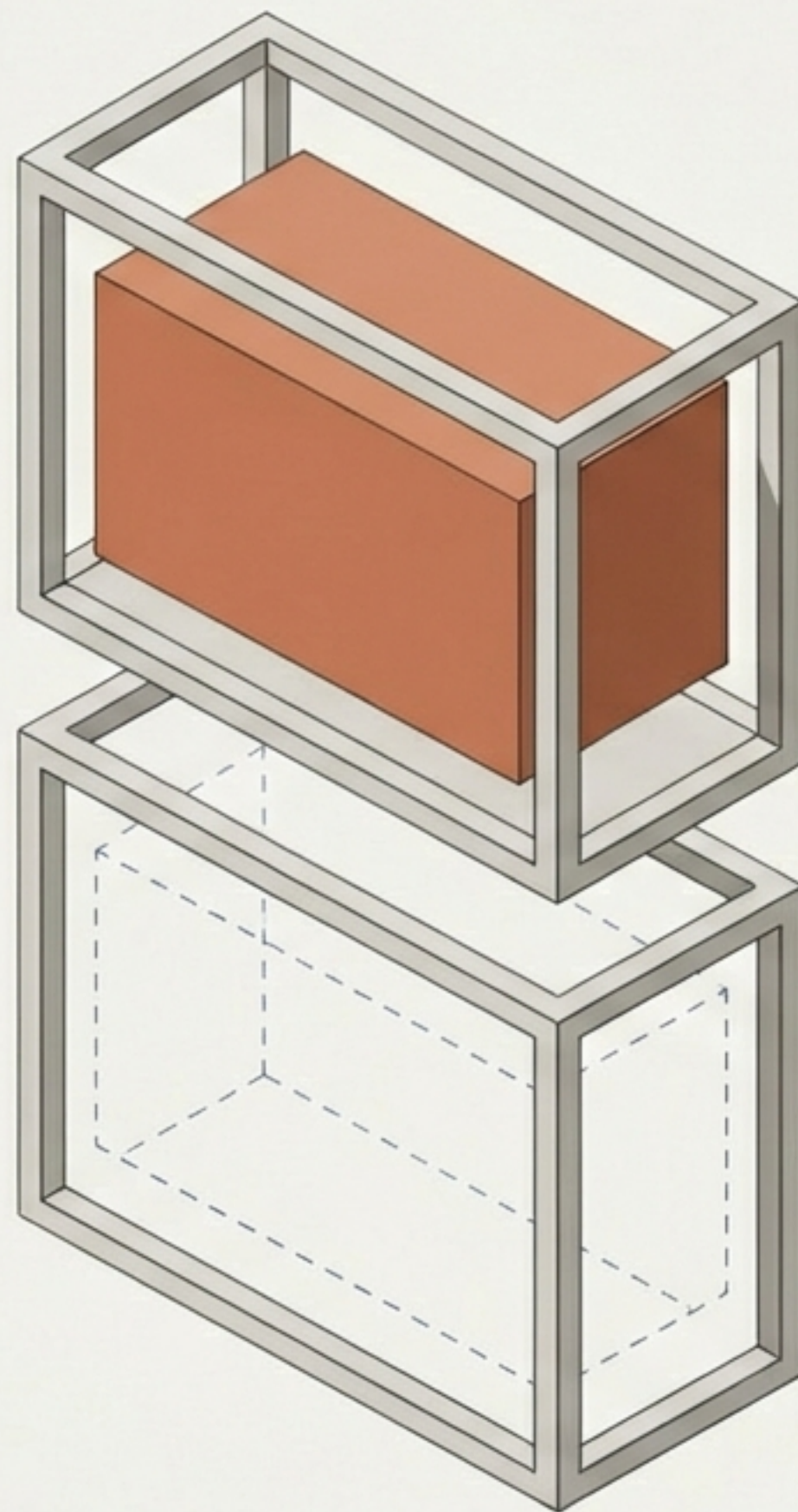


Interfaces que se Adaptam

A verdadeira arquitetura não é rígida; ela se adapta ao ambiente. Seus componentes frequentemente precisarão exibir elementos diferentes dependendo dos dados recebidos.

No React, não existem diretivas especiais de template. O fluxo de controle é feito puramente com JavaScript. Se um componente não deve exibir nada sob certas condições, ele pode simplesmente retornar `null`.

```
if (isPacked) {  
  return null;  
}  
return <Item name={name} />;
```



Lógica de Ramificação Declarativa

Para manter o JSX limpo e declarativo, utilizamos operadores do JavaScript diretamente na marcação:

Operador Ternário (? :)

`{isPacked ? <Check /> : <Cross />}`

Operador Lógico AND (&&)

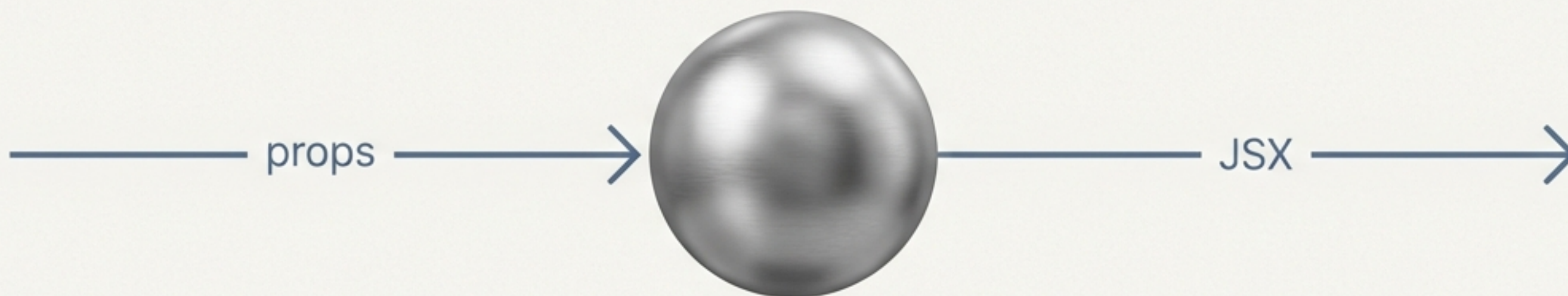
`{isPacked && <Check />}`

Atenção ao Ponto Cego:

Nunca coloque números à esquerda do &&. O JavaScript converte 0 para booleano, mas o React renderizará o número 0 na tela em vez de não renderizar nada. Sempre force um booleano: `count > 0 && <Badge />`.

A Regra de Ouro da Pureza

$$UI = f(props)$$



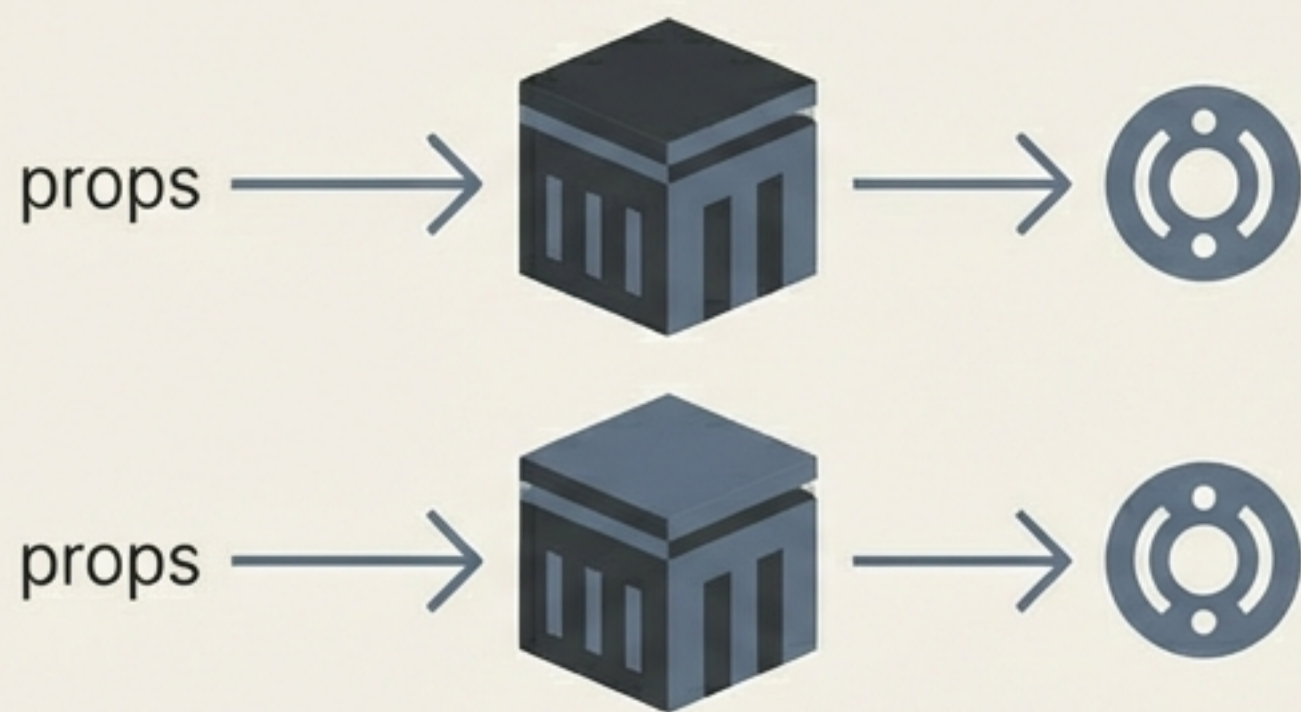
O segredo para uma arquitetura previsível e livre de bugs é tratar cada componente como uma função pura.

Na matemática, na fórmula $y = 2x$: se $x = 3$, y sempre será 6. Não depende da hora do dia ou do clima.

O React exige exatamente a mesma filosofia. Mesmas entradas (props), devem obrigatoriamente gerar a mesma saída (JSX). O componente deve cuidar da própria vida e apenas calcular a interface.

O Inimigo da Previsibilidade

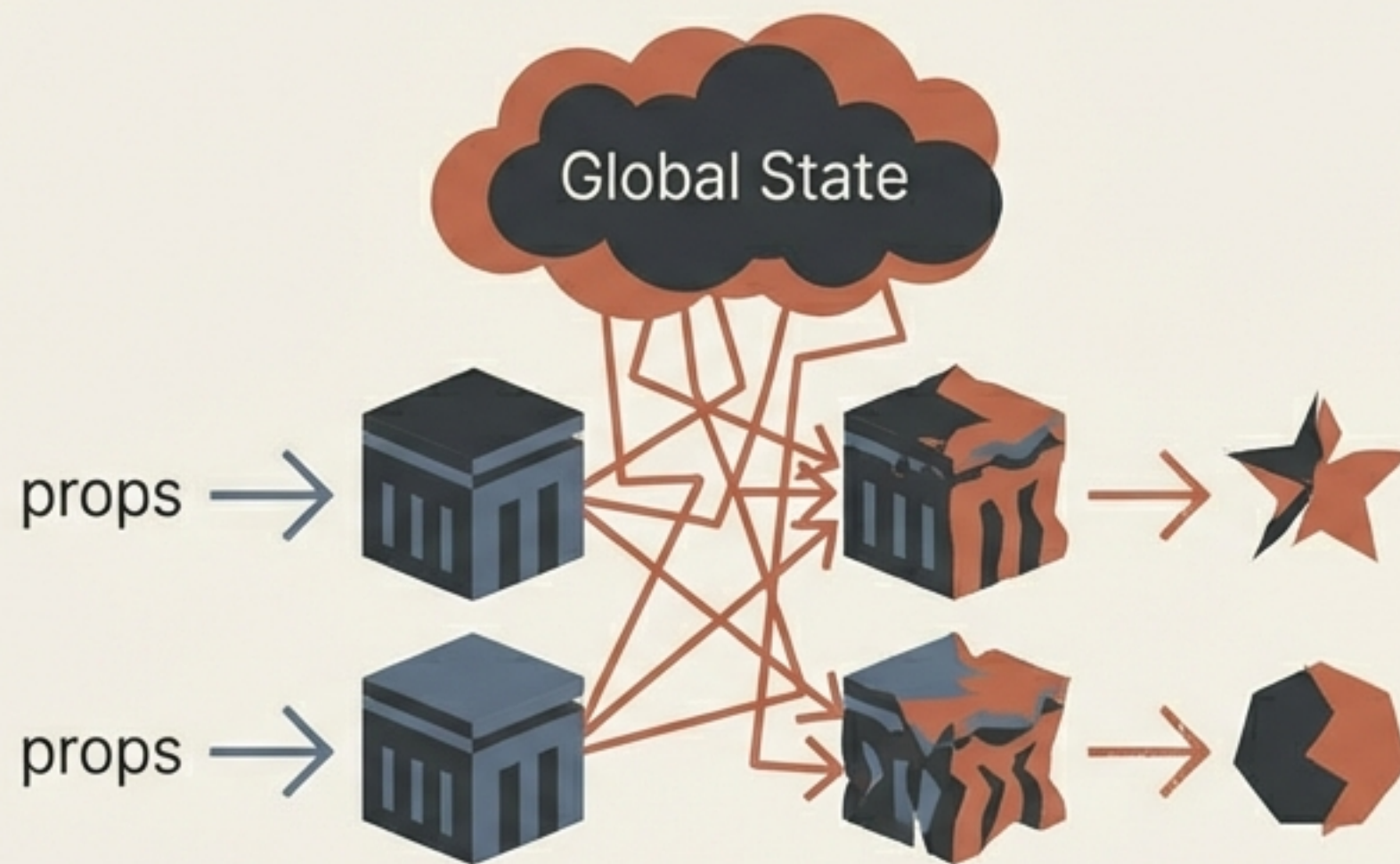
O que torna um componente impuro? Mutações externas.



Pureza

Se o seu componente alterar uma variável que existia antes dele ser renderizado, ele quebra a pureza.

Se o componente ler ou escrever em uma variável global, cada vez que ele for chamado, produzirá um JSX diferente.



Mutação Externa

Quando a pureza é quebrada, a ordem de renderização importa, o comportamento se torna caótico e componentes interferem uns nos outros.

O Segredo Aceitável da Mutação Local

A regra de pureza proíbe alterar objetos que existiam antes da renderização. No entanto, mutações de dados criados durante a própria renderização são perfeitamente normais.

Isso é chamado de **Mutação Local**. É o pequeno segredo do seu componente, totalmente contido e seguro.

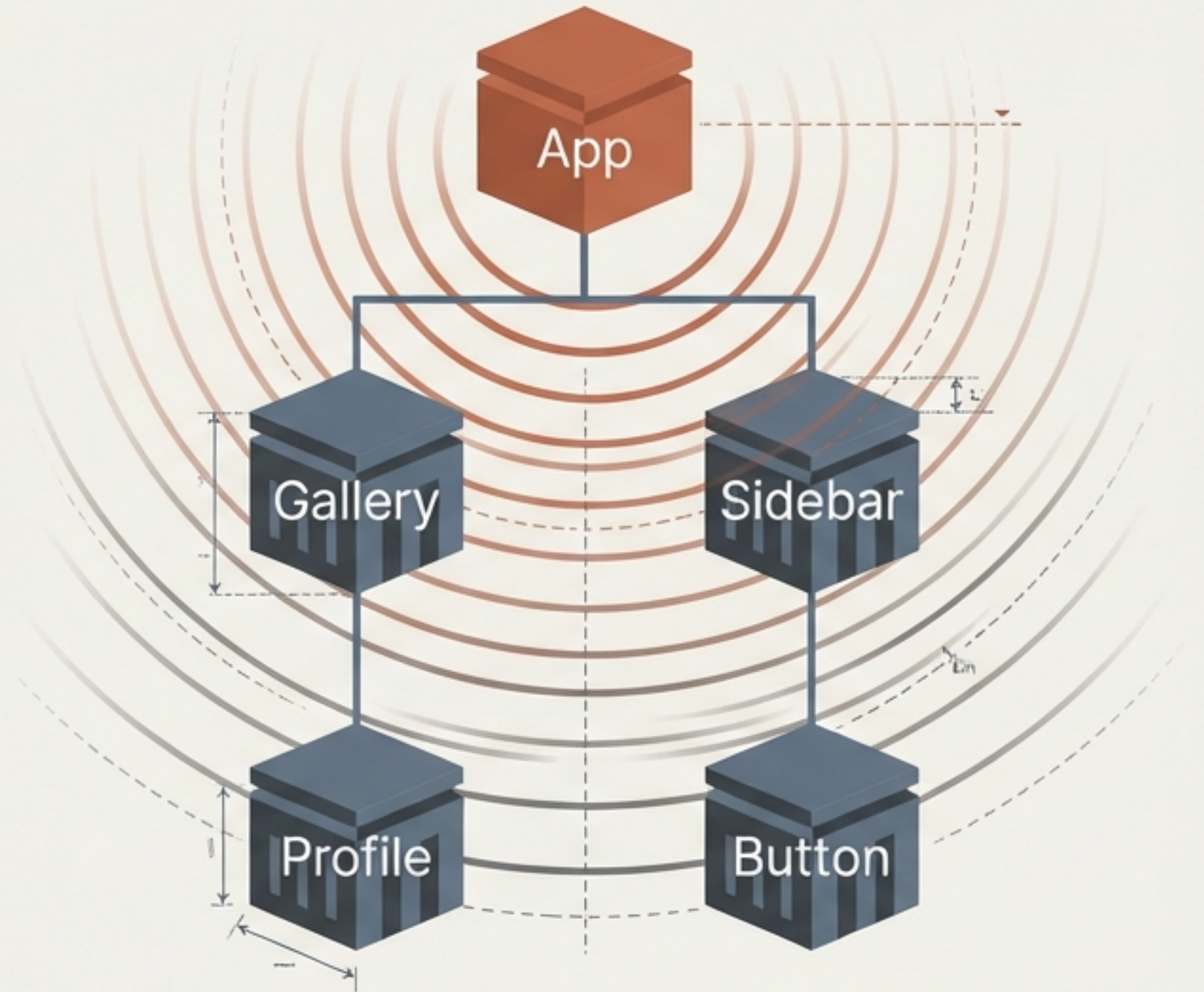
```
function Cup() {  
  let cups = [];  
  for (let i = 1; i <= 12; i++) {  
    cups.push(<Coffee key={i} />);  
  }  
  return cups;  
}
```


O Efeito Cascata das Renderizações

A árvore de componentes é reativa. O fluxo é estrutural e sempre de cima para baixo.

Quando um componente pai é re-renderizado (por receber novos dados), todos os seus componentes filhos também serão re-renderizados por padrão.

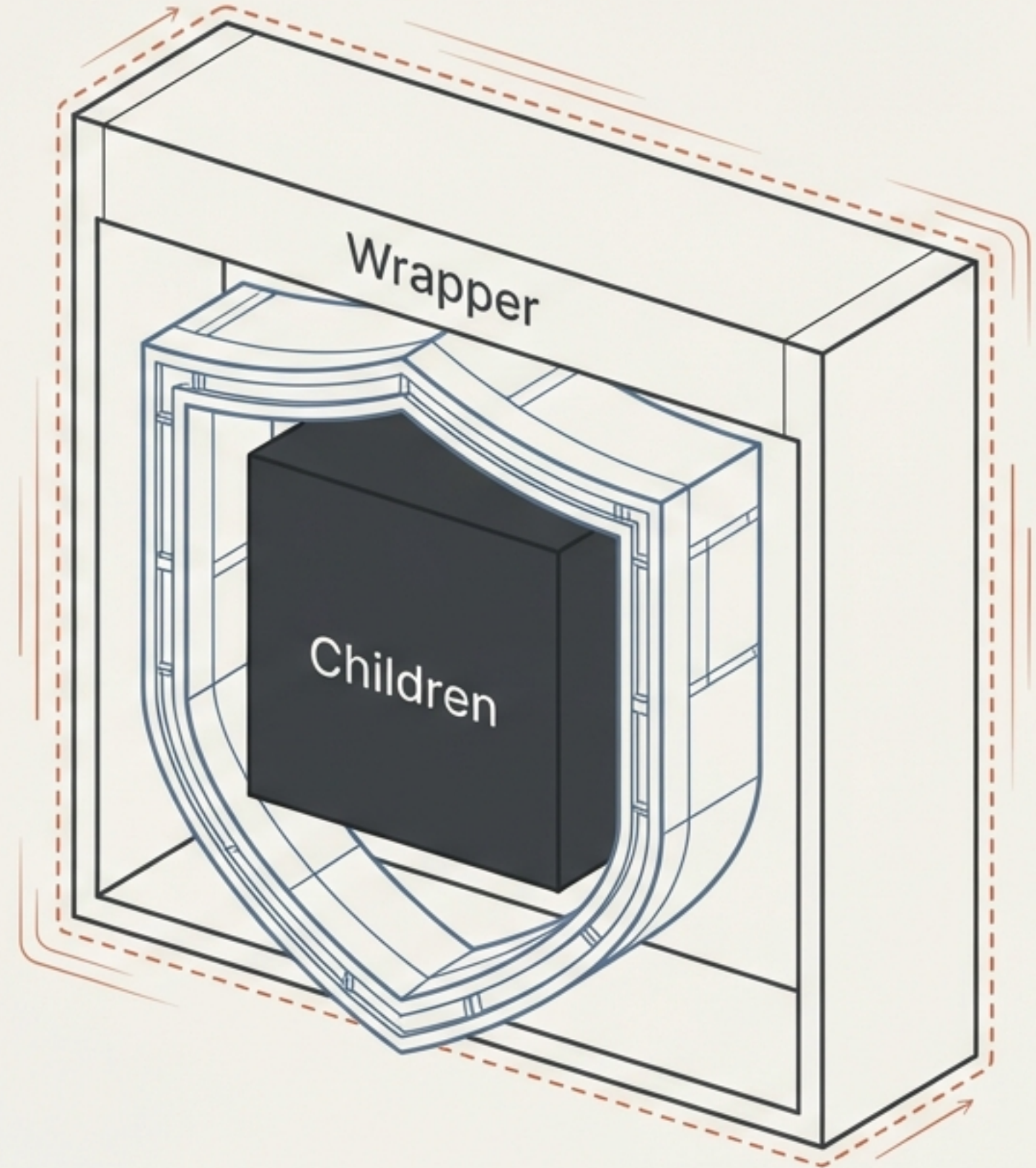
Um filho nunca força um pai a se re-renderizar diretamente. Se não controlarmos essa cascata, podemos acabar redesenhando grandes partes da interface desnecessariamente.



Otimização Através da Composição

A forma mais elegante de evitar re-renderizações desnecessárias é usar a própria arquitetura ao nosso favor. Se um contêiner recebe atualizações frequentes, mas abriga um componente pesado e estático, podemos isolá-los passando o componente pesado através da prop children.

Como as props não são afetadas pela atualização interna do invólucro, o React percebe que o children não mudou, e pula sua re-renderização.



A Filosofia do Componente Perfeito

A maestria em React não vem de ferramentas mágicas, mas do profundo respeito pela estrutura. O componente perfeito é:

- **Pequeno e Focado:** Faz apenas uma coisa e faz bem.
- **Puro e Previsível:** $UI = f(dados)$. Sem mutações externas.
- **Composicional:** Construído para se encaixar de forma flexível usando props e children.
- **Isolado:** Protege a performance da aplicação através de uma hierarquia inteligente.

Construa seus blocos com intenção, e sua interface resistirá ao teste do tempo.